

eDiscovery: Energy Efficient Device Discovery for Mobile Opportunistic Communications

Bo Han
AT&T Labs Research
Florham Park, NJ 07932, USA
Email: bohan@research.att.com

Aravind Srinivasan
Department of Computer Science and
Institute for Advanced Computer Studies
University of Maryland
College Park, MD 20742, USA
Email: srin@cs.umd.edu

Abstract—In this paper, we propose an energy efficient device discovery protocol, *eDiscovery*, as the first step to bootstrapping opportunistic communications for *smartphones*, the most popular mobile devices. We chose Bluetooth over WiFi as the underlying wireless technology of device discovery, based on our measurement study of their energy consumption on smartphones. *eDiscovery* adaptively changes the duration and interval of Bluetooth inquiry in dynamic environments, by leveraging history information of discovered peers. We implement a prototype of *eDiscovery* on Nokia N900 smartphones and evaluate its performance in three different environments. To the best of our knowledge, we are the first to conduct extensive performance evaluation of Bluetooth device discovery *in the wild*. Our experimental results demonstrate that compared with a scheme with constant inquiry duration and interval, *eDiscovery* can save around 44% energy at the expense of discovering only about 21% less peers. The results also show that *eDiscovery* performs better than other existing schemes, by discovering more peers and consuming less energy.

Index Terms—Device discovery, opportunistic communications, energy efficiency, smartphones, Bluetooth.

I. INTRODUCTION

Mobility itself is a significant problem in mobile networking. On the one hand, protocols designed for mobile networks should solve the challenges caused by the mobility of wireless devices. For example, routing protocols, such as DSR (Dynamic Source Routing) [13], are required to handle frequent routing changes and reduce the corresponding communication overhead. On the other hand, mobility can increase the capacity of wireless networks through opportunistic communications [11], where mobile devices moving into wireless range of each other can exchange information *opportunistically* during their periods of contact [4], [17].

Opportunistic communications have been widely explored in delay-tolerant networks [27], mobile social applications [17] and mobile advertising [1], to facilitate message forwarding, media sharing and location-based services. Meanwhile, there are more and more applications leveraging opportunistic communications. For example, LoKast¹ is an iPhone application that provides mobile social networking services by discovering and sharing media content among users in proximity. Nintendo

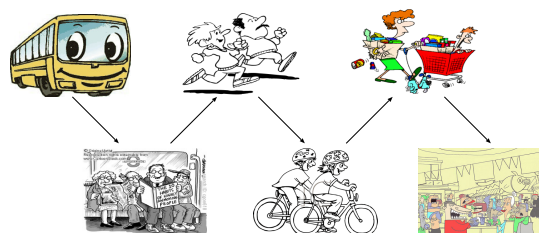


Fig. 1: Typical scenarios of opportunistic communications. A message may be exchanged on buses and subways, between joggers and bikers, and in a shopping center and a food court, before finally reaching its destination.

3DS's StreetPass² enables players to exchange game data with other users they pass on the street, through the direct device-to-device communication between 3DS systems. Other similar applications include Sony PS Vita's Near and Apple's iGroups. We show in Figure 1 several typical scenarios of mobile opportunistic communications.

Device discovery is essentially the *first* step of opportunistic communications. However, there are very few practical protocols proposed for it and most of the existing work mainly utilizes (trace-driven) simulation to evaluate the performance of various device discovery protocols [6], [25]. Moreover, although there are several real-world mobility traces in the CRAWDAD repository³ which were collected using Bluetooth device discovery, most of them used very simple discovery protocols with fixed inquiry duration and interval. A recently proposed opportunistic Twitter application [20] also uses a 2-minute inquiry interval for Bluetooth device discovery. It is known that these kinds of discovery protocols are not energy efficient [25] and thus may not be desirable for power-constrained mobile devices, such as smartphones. In this paper, we bridge this gap by developing an *energy-aware* device discovery protocol for smartphone-based opportunistic communications and evaluating its performance in practice.

There are two major challenges in designing, implementing and evaluating energy efficient device discovery protocols for smartphones. First, the selection of underlying communication

¹<http://www.lokast.com/>

²<http://www.nintendo.com/3ds/features/>

³<http://crawdad.cs.dartmouth.edu/>

technology is complicated by the multiple wireless interfaces on smartphones, such as Bluetooth and WiFi (a.k.a., IEEE 802.11).⁴ Although Bluetooth is a low-power radio, its device discovery duration is much longer than WiFi (~ 10 s for Bluetooth vs. ~ 1 s for WiFi active scanning), which may cause more energy consumption on smartphones. Similarly, WiFi is known to be power-hungry for mobile devices [18], [22]. Thus, it is not clear which of them is more suitable for device discovery on smartphones.

Second, given the dynamic nature of human mobility, we need to adaptively tune the parameters of device discovery, such as inquiry duration and interval, to reduce smartphone energy consumption. Schemes with constant inquiry intervals have been proven to be optimal in terms of minimizing discovery-missing probability [25]. However, their energy consumption is usually higher than the adaptive ones, which may miss more devices during discovery procedures. Therefore, we have a tradeoff between energy consumption and discovery-missing probability.

We make the following contributions in this paper.

- We present a systematic measurement study of the energy consumption of Bluetooth and WiFi device discovery on smartphones, by measuring both the electrical power and the discovery duration (Section IV). Based on our measurement results, we chose Bluetooth as the underlying wireless technology. Previous work has studied the power of Bluetooth/WiFi devices [6], [8], [18]. However, they either focus on only Bluetooth [6] or ignore the duration of device discovery [8], [18], without which it is hard to evaluate the energy consumption of these devices.
- We design an energy-aware device discovery protocol, named *eDiscovery*, as the first and very important step to bootstrapping smartphone-based opportunistic communications (Section V). By trading energy consumption for a limited discovery loss, we demonstrate that *eDiscovery* is highly effective in saving energy on smartphones. *eDiscovery* dynamically tunes the discovery duration and interval according to history information of the number of discovered peers. It also introduces randomization into device discovery, in order to explore the search space further.
- Our major contribution is an extensive performance evaluation of *eDiscovery* and other existing device discovery protocols in different realistic environments, through a prototype implementation on Nokia N900 smartphones (Section VI). We conduct experiments in a university campus, a metro station and a shopping center. Our experimental results verify the effectiveness of *eDiscovery* in practice. Compared with the STAR protocol proposed by Wang et al. [25], *eDiscovery* consumes less energy and discovers more peers. *eDiscovery* also performs better than another protocol in the literature.

⁴We prefer Bluetooth and WiFi to 3G, as they are local communication technologies with almost no monetary cost.

II. RELATED WORK

In this section, we briefly review device discovery in wireless networks and mobile opportunistic communications.

A. Wireless Device Discovery in General

Device discovery has been widely studied in various wireless networks, such as mobile opportunistic networks [6], [12] and delay-tolerant networks [25].

Neighbor/device discovery is one of the first steps to initialize large wireless networks. McGlynn and Borbash [16] examine the problem of neighbor discovery during the deployment of static ad-hoc networks, where the discovery may last only a few minutes. Inspired by the birthday paradox, a pair of nodes perform neighbor discovery by transmitting and listening on k independently and randomly chosen slots among n slots (the ratio k/n is relatively small). Vasudevan et al. [24] show that an existing ALOHA-like neighbor discovery algorithm reduces to the classical Coupon Collector's Problem when nodes are not capable of collision detection. They also propose an improved algorithm based on receiver status feedback when nodes have a collision detection mechanism. Differently from the above works that are based on abstract communication models, our focus is practical Bluetooth device discovery for smartphone-based opportunistic communications.

Dutta and Culler [7] propose an asynchronous neighbor discovery protocol, called Disco, for mobile sensing applications. Disco can address the challenge of operating the radios at a low duty cycle and ensuring fast and reliable discovery in bounded time through the adaptation of the Chinese Remainder Theorem. U-Connect [14] is another asynchronous neighbor discovery protocol for mobile sensor networks that selects carefully the time slots to perform discovery and that has been proven theoretically better than Disco. The above works in sensor networks aim to achieve a trade-off between discovery latency and energy consumption.

The goal of *eDiscovery* is similar in spirit to that of Wang et al. [25] which investigates the trade-off between the contact probing frequency (which determines energy consumption) and the missing probability of a contact for delay tolerant applications. They also design a contact probing algorithm, named STAR (Short Term Arrival Rate), to dynamically change the contact probing frequency based on the contact arrival process. Without specifying the communication technologies, they assume that every probing message is just an impulse and consumes no time. We compare the performance of *eDiscovery* with STAR in Section VI through extensive real-world experiments.

B. Bluetooth Device Discovery

Bluetooth specifies a detailed device discovery protocol [2]. Salonidis et al. [21] identify the bottlenecks of asymmetric device-discovery delay of Bluetooth. They introduce a randomized symmetric discovery protocol to reduce this delay. Based on Bluetooth specification v1.1, Peterson et al. [19] derive rigorous expressions for the inquiry-time probability distribution of two Bluetooth devices that want to discover

each other and validate them through simulation studies. Chakraborty et al. [3] present an analytical model of the time of Bluetooth device discovery protocol. They investigate the discovery time pattern through extensive simulation studies.

Liberatore et al. [15] solve the problem of long discovery duration of Bluetooth due to its half-duplex discovery process by the addition of another Bluetooth radio. Through analysis and simulation studies they demonstrate that this dual radio technique can improve both discovery duration and connection frequency. Drula et al. [6] study how to select Bluetooth device discovery parameters according to the mobility context and thus reduce the energy consumption of device discovery. They present two algorithms that adjust these parameters based on recent activities and the location of previous contacts, and evaluate their performance through simulations. In our previous work [12], we compare energy consumption of Bluetooth and WiFi device discovery on Nokia N900 smartphones, using battery life as a metric. We evaluate the Bluetooth device-discovery probability in an office environment using a static phone and a moving phone.

Besides the above works, although there is a large body of literature about Bluetooth device discovery, most of them focus on the improvements of discovery latency between two Bluetooth devices by tuning various parameters or changing the protocol itself, which may not be feasible to implement on smartphones. Differently from them, we study how to dynamically change the inquiry window and interval to achieve the trade-off between discovery-missing probability and energy efficiency. Particularly, we design an energy-aware Bluetooth device discovery protocol and evaluate its performance *in the wild* through a prototype implementation on smartphones.

C. Opportunistic Communications

There have been many applications of opportunistic communications in mobile social networks and delay-tolerant networks. To encourage social participation from mobile users in information sharing applications, Garyfalos and Almeroth [9] propose Coupons, an incentive scheme that allows users to opportunistically share data over a wireless medium. Previously, we have proposed to leverage opportunistic communications and social participation to offload cellular traffic to mobile-to-mobile communications and thus alleviate traffic load on 3G networks [12]. The above works can benefit from our proposed scheme to facilitate their opportunistic communications.

McNamara et al. [17] propose a content source selection scheme, Media Sharing, to share media content among co-located mobile users in urban transport. With this scheme, mobile devices can select the best content sources (the peers who can remain co-located long enough to complete data transfer) and perform content sharing and distribution. The authors confirm the feasibility of the proposed prediction scheme using underground transport traces collected from a large metropolitan mass transit system. Differently from Media Sharing, we aim to develop an energy efficient device discovery protocol, which is an essential step *before* the selection of the best peers.

III. DEVICE DISCOVERY IN BLUETOOTH AND WiFi

In the following, we discuss device discovery of Bluetooth and WiFi, the two most commonly available local wireless communication technologies on smartphones.

A. Bluetooth

The Bluetooth specification (Version 2.1) [2] defines all layers of a typical network protocol stack, from the baseband radio layer to the application layer. Bluetooth operates in the 2.4 GHz ISM (Industrial, Scientific and Medical) frequency band, shared with other devices such as IEEE 802.11 stations, baby monitors and microwave ovens [10]. Therefore, it uses Frequency-Hopping Spread Spectrum (FHSS) to avoid cross-technology interference, by randomly changing its operating frequency bands. Bluetooth has 79 frequency bands (1 MHz width) in the range 2402-2480 MHz and the duration of a Bluetooth time slot is 625 μ s. In the following we focus on device discovery and refer interested readers to Smith et al. [23] for further study of the Bluetooth protocol stack.

During device discovery, an inquiring device sends out inquiry messages periodically and waits for responses, and a scanning device listens to wireless channels and sends back responses after receiving inquiries [2]. The inquiring device uses two trains of 16 frequency bands each, selected from 79 bands. The 32 bands of these two trains are selected according to a pseudo-random scheme and a Bluetooth device switches its trains every 2.56 seconds. In every time slot, the inquiring device sends out two inquiry messages on two different frequency bands and waits for response messages on the same frequency bands during the next time slot. After a device receives an inquiry message, it will wait for 625 μ s (i.e., the duration of a time slot) before sending out a response message on the same frequency band, which completes the device discovery procedure. For scanning devices, Bluetooth controls their scanning duration and frequency with two parameters, scan window and scan interval.

B. WiFi

The key concept of device discovery in WiFi is well understood. WiFi stations in infrastructure and ad-hoc modes periodically (100 ms by default) send out Beacon messages to announce the presence of a network. A Beacon message includes information such as SSID (service set identifier) and capability information. The WiFi interfaces of mobile phones should operate in ad-hoc mode and form an Independent Basic Service Set (IBSS) to support opportunistic communications, since infrastructure-mode interfaces cannot form a network and thus cannot communicate directly. Besides sending out Beacon messages, a WiFi interface also scans wireless channels to discover peers.

There are two types of WiFi scanning, passive and active. In passive scanning, a WiFi interface listens for Beacon messages on each channel, broadcasted by its peers at regular intervals. It periodically switches channels, but does not send any probe request message. During active scanning, a WiFi interface actively searches for its peers, by broadcasting probe request

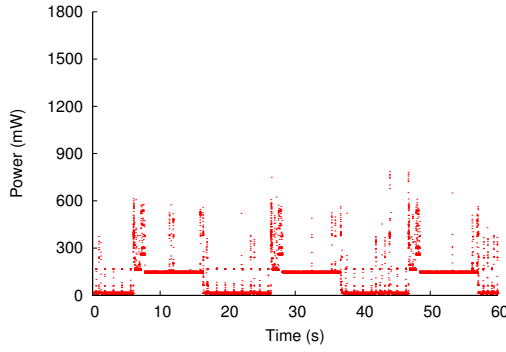


Fig. 2: A 60-second snapshot of the temporal power of periodic Bluetooth device discovery with 10-second interval. The smartphone under test is a Nokia N900 smartphone.

messages on each possible operating channel (channels 1 to 11 in North America). It then waits for probe response messages from its peers, which include information similar to that in Beacon messages.

We prefer active scanning to passive scanning for device discovery of opportunistic communications mainly for two reasons. First, although passive scanning has the advantage of not broadcasting probe request messages, it dwells on each channel longer than active scanning, to collect Beacon messages from peers, and thus may consume more energy. Second, an ad-hoc mode interface may skip the sending of Beacon messages and thus make itself not discoverable by passive scanning, when it tries to scan for other peers with the same SSID (which happens frequently when it is the only station in an IBSS).

IV. ENERGY CONSUMPTION OF DEVICE DISCOVERY

In this section, we measure the power and energy consumption of Bluetooth and WiFi device discovery on smartphones. Based on the experimental results, we chose Bluetooth as the communication technology for smartphone-based device discovery. Although previous work has measured energy consumption of WiFi and Bluetooth devices several years ago [6], [18], these results may be invalid given the rapid development of battery and wireless technologies [8]. To the best of our knowledge, there is no systematic study of smartphone energy consumption of Bluetooth and WiFi device discovery.⁵

A. Measurement Setup

We measure the electrical power of two states of Bluetooth and WiFi device discovery, idle and active probing, on Nokia N900 smartphones using the Monsoon power monitor⁶. The default OS of Nokia N900, Maemo 5, is an open source Linux distribution (kernel version 2.6.28). Its WiFi chipset is Texas

⁵Although Friedman et al. [8] have recently studied the power of Bluetooth scanning and WiFi search, they overlook the duration of device discovery which also affects the energy consumption. Furthermore, their measurements are for station mode WiFi interfaces and demonstrate inconsistent results about WiFi device discovery.

⁶<http://www.monsoon.com/LabEquipment/PowerMonitor/>

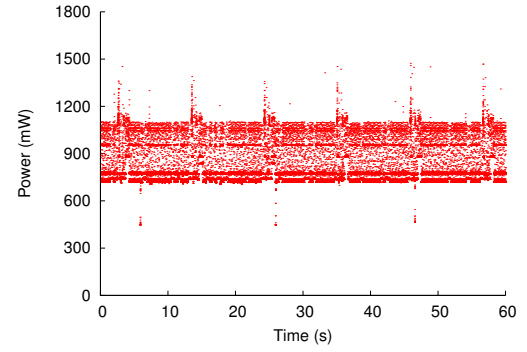


Fig. 3: A 60-second snapshot of the temporal power of periodic WiFi device discovery with 10-second interval. The smartphone under test is a Nokia N900 smartphone.

# of Devices	Average	Standard Deviation
0	162.03	2.12
1	227.06	12.33
2	247.72	8.60
3	248.91	9.51
4	248.59	3.16
5	256.02	4.93
6	253.05	5.51

TABLE I: The electrical power (in mW) of Bluetooth device discovery with different numbers of neighboring devices.

Instruments WL1251 using the wl12xx device driver⁷. Its Bluetooth chipset is Broadcom BCM2048. We use BlueZ⁸, the default Bluetooth protocol stack of most Linux distributions, to run Bluetooth device discovery experiments. During the measurements, we redirect standard output to `\dev\null` and turn the screen off to minimize their impact on the measurement results. We report the average result and standard deviation for each configuration over 10 runs in this section.

B. Bluetooth

We present a 60-second snapshot of the power of Bluetooth device discovery in Figure 2. We perform the experiments by running `hcitool`, a tool that can send commands, such as `inq` (inquiry), to Bluetooth devices. We use the `flush` option to clear the cache of previously discovered devices before each inquiry. During the measurements, the phone queries neighboring Bluetooth devices periodically with a 10-second interval. When there is no neighboring device, the average power of Bluetooth inquiry over 10 runs is ~ 162.03 mW (standard deviation: 2.12 mW). During inquiry intervals (i.e., idle states), the Bluetooth radio is in discoverable mode with average power ~ 16.54 mW (standard deviation: 1.11 mW). Note that all results of power measurements in this paper include the baseline power of the smartphone under test.

The average power of Bluetooth device discovery is affected by the number of neighboring devices. We repeat

⁷<http://linuxwireless.org/en/users/Drivers/wl12xx>

⁸<http://www.bluez.org/>

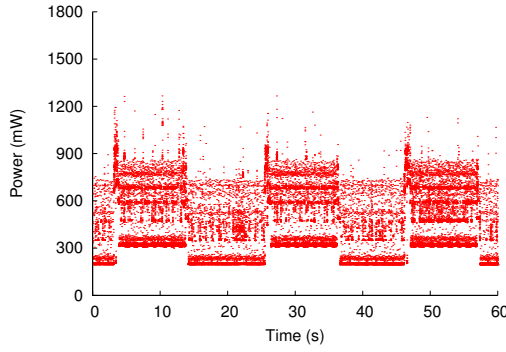


Fig. 4: A 60-second snapshot of the temporal power of periodic Bluetooth device discovery with 10-second interval. The smartphone under test is a HTC Hero smartphone (Android 1.5).

Environment	Office	Home	Park
# of peers	43.52 (5.3)	14.02 (1.4)	0.01 (0.1)
duration (s)	1.07 (0.15)	0.87 (0.05)	0.52 (0.04)

TABLE II: The average number of discovered peers and duration of WiFi device discovery in three environments. The numbers in the parentheses are the standard deviations.

the experiments with the number of neighboring Bluetooth devices increasing from 0 to 6 and summarize the results in Table I. As we can see from this table, when there is one neighboring device, the average power increases to around 227.06 mW, due to the reception of response messages of Bluetooth inquiry. When there are more than one neighboring devices, the average power increases to about 250 mW.

Defined in the standard [2], the duration of Bluetooth device discovery should be a multiple of 1.28 seconds and the recommended default value is 10.24 seconds, which we used in the measurements. Figure 2 shows clearly the configured Bluetooth device discovery duration and interval.

C. WiFi

We present another 60-second snapshot of the power of WiFi device discovery in Figure 3. We perform the experiments by running `iwlist`, a tool that shows the list of access points and ad-hoc cells in range through active scanning. During the measurements, the phone scans neighboring devices periodically also with a 10-second interval, which can be clearly identified in Figure 3. The average power of WiFi active scanning over 10 runs is ~ 836.65 mW (standard deviation: 8.98 mW). Even during scanning intervals, the average power is ~ 791.02 mW (standard deviation: 5.23 mW), because the WiFi radio is in ad-hoc mode and sends out Beacon messages with 100 ms intervals.

Differently from Bluetooth, the duration of WiFi active scanning is not fixed and may depend on the number of operation channels and the amount of neighboring peers. We measure the duration of WiFi device discovery in three different environments: a campus office building, an apartment, and a national park, and summarize the results in Table II. In each environment, we repeat the experiments 100 times

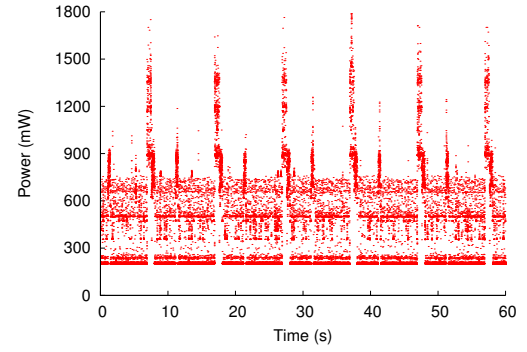


Fig. 5: A 60-second snapshot of the temporal power of periodic WiFi device discovery with 10-second interval. The smartphone under test is a HTC Hero smartphone (Android 1.5).

	P_{idle}	P_{probe}
Bluetooth	16.54 (1.11)	253.05 (5.51)
WiFi	791.02 (5.23)	836.65 (8.98)

TABLE III: The average power of Bluetooth and WiFi device discovery in mW.

and report the average values and standard deviations. As we can see from this table, when the number of discovered peers increases, the duration of WiFi device discovery grows from ~ 0.52 seconds to ~ 1.07 seconds, which is much shorter than the duration of Bluetooth inquiry.

D. Energy Consumption

We summarize the average power of Bluetooth (with 6 neighboring devices) and WiFi device discovery in Table III. Suppose the power is P_{idle} for the idle state and P_{probe} for the inquiry/scan state of Bluetooth/WiFi devices, the duration of Bluetooth inquiry/WiFi scan is T_{probe} and the inquiry/scan interval is T_{idle} . Then the estimated energy consumption is

$$E = T_{idle} \cdot P_{idle} + T_{probe} \cdot P_{probe}$$

Given the high power of WiFi device discovery in both active probing and idle states, we prefer Bluetooth to WiFi for device discovery of smartphone-based opportunistic communications. We note that no matter how long the duration of Bluetooth inquiry is, the overall energy consumption of Bluetooth device discovery should always be lower than that of WiFi, because the power of Bluetooth inquiry is even lower than that of the WiFi idle state (253.05 vs. 791.02 mW). To perform device discovery, the major problem of WiFi ad-hoc mode is that the radio needs to send out Beacon messages periodically and power saving mechanisms for WiFi ad-hoc mode are not available on most mobile phones [22].

Although the communication range of WiFi is longer than Bluetooth and may discover more peers, making its device discovery energy efficient requires substantial modifications of the WiFi protocol, which may not be feasible on most smartphones. In this paper, we aim to design a device discovery protocol without changing the underlying communication protocol and thus make its deployment easy.

E. Android Smartphones

We also measured the power of Bluetooth and WiFi device discovery using a HTC Hero smartphone with Android 1.5. We plot the results in Figure 4 for Bluetooth and Figure 5 for WiFi. On this smartphone, the average power is 432.84 mW (standard deviation: 7.86 mW) for Bluetooth inquiry and 900.25 mW (standard deviation: 21.54 mW) for WiFi scan. There are two differences of the experiments on the Nokia N900 and HTC Hero smartphones. First, the experiments on HTC Hero were performed with the screen on due to the operational requirements and thus the baseline power of HTC Hero is higher than that of Nokia N900. Second, the WiFi interface on HTC Hero does not support ad-hoc mode and we cannot measure the average power P_{idle} on it. However, these results still clearly show the significant power difference (467.41 mW) of Bluetooth inquiry and WiFi scan.

V. eDISCOVERY DESIGN

In this section, we present eDiscovery, an energy-aware device discovery protocol that adaptively changes the duration of Bluetooth inquiry and the probing interval.

The major design principle of eDiscovery is to reduce smartphone energy consumption of device discovery, while not missing too many peers. To achieve this goal, we dynamically change the duration and interval of Bluetooth device discovery, based on the number of discovered peers. We present the adaptive inquiry algorithm of eDiscovery in Algorithm 1.

There are two approaches to control the duration of Bluetooth device discovery: (1) specifying the length of the inquiry window explicitly or (2) specifying the number of received responses before device discovery stops. Accordingly, there are two parameters of `hci_inquiry`, the device discovery function of BlueZ, `inquiry_window` and `num_responses`. This function stops inquiry after $1.28 \times \text{inquiry_window}$ seconds or it has received `num_responses` inquiry responses.

We focus on the control of the inquiry window in this paper, as it is hard to predict the number of neighboring peers in practice. Moreover, a peer can respond to an inquiry more than once. Suppose there are 3 neighboring peers, A, B, and C, and we set `num_responses` to be 3. If all the first 3 responses are sent by peer A, device discovery will stop after receiving them and thus discover only peer A.

The two key parameters in Algorithm 1 of eDiscovery are `inquiry_window` and `inquiry_interval`, which control the duration and interval of Bluetooth inquiry. We initialize `inquiry_window` to be 8 (i.e., $8 * 1.28 = 10.24$ seconds, the default standard value) and `inquiry_interval` to be 10 seconds. We set `MAX_RSP` to be 255, the maximum allowed value. The main body of this algorithm is a while loop that performs Bluetooth inquiry $1.28 * \text{inquiry_window}$ seconds and then sleeps `inquiry_interval` seconds.

After each Bluetooth inquiry, we adapt the values of `inquiry_window` and `inquiry_interval` based on the number of discovered peers. If this number is larger than N , we keep the default value 8, aiming to discover more peers. If it is smaller

Algorithm 1 Adaptive Inquiry Algorithm of eDiscovery

```

1: inquiry_window = 8, inquiry_interval = 10;
2: while (TRUE) do
3:   peers = hci_inquiry(inquiry_window, MAX_RSP);
4:   if (peers >  $N$ ) then
5:     inquiry_window = 8;
6:   else
7:     inquiry_window = 5 +  $r$ ;
8:   end if
9:   if (peers == 0 and last_peers == 0) then
10:    inquiry_interval += 10 +  $r$ ;
11:   else if (peers <> 0 and last_peers == 0) then
12:    inquiry_interval = 10 +  $r$ ;
13:   else if (peers > last_peers) then
14:    inquiry_interval -=  $I$ ;
15:   else if (peers < last_peers) then
16:    inquiry_interval +=  $I$ ;
17:   end if
18:   last_peers = peers;
19:   sleep(inquiry_interval);
20: end while

```

or equal to N , we set the next `inquiry_window` to be $5 + r$, where r is defined as

$$r = \begin{cases} 1 & \text{with probability 0.1} \\ 0 & \text{with probability 0.8} \\ -1 & \text{with probability 0.1} \end{cases}$$

We chose $5 + r$ to make the smallest `inquiry_window` 4, because this is the minimum inquiry window to perform a complete scan of all possible frequency bands. Moreover, Peterson et al. [19] demonstrate that by setting the `inquiry_window` to be 4, a Bluetooth device can locate 99% of neighboring devices within its transmission range in a *static* environment. By changing `inquiry_window` in this way, we can reduce the duration of Bluetooth inquiry and thus save energy on smartphones when the number of neighboring peers is small.

We adapt the value of `inquiry_interval` to the number of discovered peers in a similar way. When a smartphone discovers no peers for two consecutive inquiries, we increase `inquiry_interval` by $10 + r$ and reset it to $10 + r$ after the smartphone discovers new peers. Moreover, if the current number of discovered peers is larger than the previous one, we decrease `inquiry_interval` by I , and vice versa. An implication of this algorithm is that `inquiry_interval` will not change if the number of discovered peers does not vary. We allow `inquiry_interval` to vary between 10 – 200 seconds.

This random variable r is refreshed for every inquiry. We use it for improving the robustness of eDiscovery for dynamic environments. Furthermore, it can avoid synchronization of Bluetooth inquiry which may make Bluetooth devices not be able to discover each other [12], [19]. The choice of the probabilities in r is not arbitrary. Essentially, we want `inquiry_window` to be 5 and `inquiry_interval` to be 10 under certain conditions with a high probability and slightly change

their values by 1 with a low probability.

The intuition behind these adaptations is that we can reduce the inquiry duration and increase the inquiry interval when the number of neighboring peers is small. By changing the constants N and I , we can achieve different trade-off between the number of discovered peers and smartphone energy consumption. Smaller N and I lead to more aggressive Bluetooth inquiry, which may discover more peers but also consume more energy on smartphones. We evaluate the performance of eDiscovery with different combinations of N and I in the next section.

VI. PERFORMANCE EVALUATION

In this section, we evaluate the performance of our proposed eDiscovery protocol through a prototype implementation on Nokia N900 smartphones and compare it with other schemes. Although previous work has evaluated device discovery protocols using simulations [6], [25], a recent study demonstrates that even contact-based simulations using real-world mobility traces may not be able to accurately evaluate the performance of opportunistic networks [20]. Moreover, it is also not clear how Bluetooth device discovery performs in the wild, under cross-technology interference [10].

We implement eDiscovery in C language using the BlueZ protocol stack and compare its performance with three other approaches: a scheme with constant inquiry interval (referred as Constant in the following), the STAR algorithm by Wang et al. [25] and the Recent Activity Level (RAL) scheme by Drula et al. [6]. The two metrics we are interested in are the ratio of discovered peers, compared to the ground truth, and the estimated energy consumption on smartphones. To get the ground truth, we perform Bluetooth inquiry with the default 10.24-second duration continuously. Based on the ground truth, we can know how many peers Constant can discover by aggregating the inquiries in the ground truth with only odd/even indices. We did all experiments three times and report the average results with standard deviations.

A. Impact of N and I

We first evaluate the performance of eDiscovery for different combinations of N and I , using Constant as the baseline. During a single experiment, we run the continuous Bluetooth inquiry on one phone and eDiscovery on another simultaneously. We chose to use only two phones intentionally for reducing the possible collisions of Bluetooth inquiry messages from the experimental phones. We conducted the experiments in and around the Stamp Student Union of the University of Maryland. We walked along a pre-defined route for around 30 minutes during the experiments. Most of the Bluetooth devices discovered by us should be on mobile phones, although they can also be on other mobile devices such as tablets and laptops.

We plot the percentage of discovered Bluetooth devices of eDiscovery and Constant in Figure 6. We also summarize their estimated energy consumption in Table IV. The experimental results show that increasing N and I can save

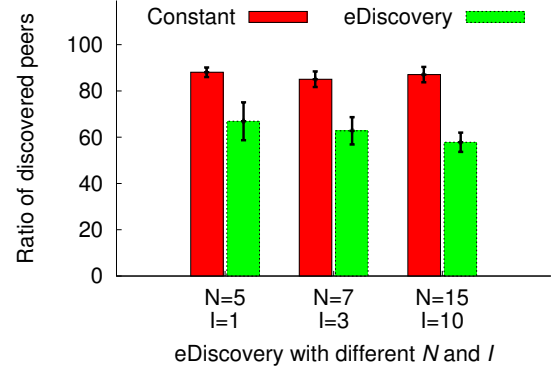


Fig. 6: The ratio of the number of discovered peers for Constant and eDiscovery to the ground truth, with different N and I .

Parameters	Constant	eDiscovery	Percentage
$N = 5, I = 1$	220.83 (7.21)	123.93 (7.08)	56.12%
$N = 7, I = 3$	209.02 (5.65)	113.84 (15.55)	54.46%
$N = 15, I = 10$	210.80 (8.78)	105.60 (1.68)	50.09%

TABLE IV: The estimated energy consumption (in Joules) of eDiscovery with different N and I and the comparison with Constant.

smartphone energy consumption at the expense of a higher missing probability. When $N = 5$ and $I = 1$, eDiscovery consumes only 56% energy of Constant, and discovers 21% less peers than it. These results also partially verify experimentally the theoretical analysis by Wang et al. [25] that the probing scheme with constant inquiry intervals achieves the minimum discovery-missing probability among all probing methods with the same average inquiry interval. The ratio of discovered peers between Constant and the ground truth is higher than 80% for all experiments.

B. Dynamic Environment

We then compare the performance of eDiscovery ($N = 5$ and $I = 1$) with Constant and STAR [25] in three different environments: the Student Union of the University of Maryland, the Union Station of Washington D.C. and the Mall at Short Hills in New Jersey. We also chose a pre-defined route in the other two locations, including both indoor and outdoor environments, and the duration of experiments was about 30 minutes too. Generally, there are much more peers in the indoor environment than the outdoor environment in these three locations. We limit the inquiry interval of STAR to be 10 – 200 seconds, the same as eDiscovery.

We plot in Figure 7 the percentage of discovered peers of eDiscovery, Constant and STAR, compared with the ground truth. We also plot in Figure 8 the energy consumption of eDiscovery and STAR, compared with Constant. As we can see from these figures, eDiscovery performs better than STAR in all three locations. In particular, eDiscovery discovers more peers than STAR but consumes much less energy on smartphones.

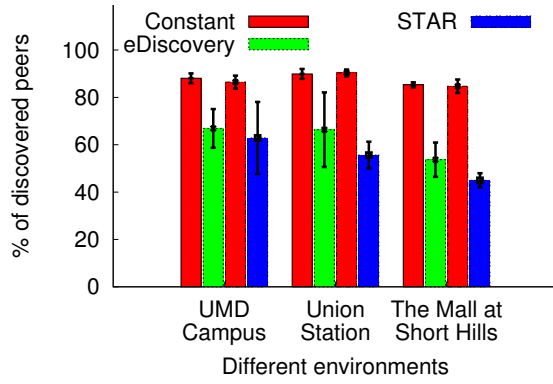


Fig. 7: The comparison of the percentage of discovered peers for different schemes.

eDiscovery outperforms STAR for the following three reasons. First, the adaptive inquiry algorithm of eDiscovery is very simple yet effective. The design of STAR relies on the characteristics of a contact trace collected in Singapore, such as the contact duration distribution. Although it may work well for that specific trace, its performance may not be guaranteed for other environments.⁹ Second, eDiscovery takes into account not only the inquiry interval, but also the duration of inquiry, to further reduce smartphone energy consumption. As shown in Section IV, the active probing state consumes much more energy than the idle state of Bluetooth inquiry. Third, it adapts to environmental changes (i.e., the number of neighboring peers) much more quickly than STAR, which is important in dynamic environments.

C. An In-Depth Look at the Traces

To verify the above, we took an in-depth look at the traces collected for the experiments we did in the Mall at Short Hills. We plot the start time, duration, and the number of discovered peers of a single experiment for STAR in Figure 9a and for eDiscovery in Figure 9b. For a Bluetooth device discovery starting at s and ending at t that discovers p peers, we plot a horizontal bar from (s, p) to (t, p) . We note that in these two figures, a Bluetooth device may be counted several times if it appeared in multiple device discoveries. In each figure, we use the red color to plot the ground truth and the black color for either STAR or eDiscovery. During both experiments, we discovered more than 100 peers in the ground truth. The percentage of discovered peers is around 60% for eDiscovery and 40% for STAR.

There are two main observations from Figure 9a and Figure 9b. First, on average the duration of Bluetooth device discovery in eDiscovery is shorter than STAR (6.79 seconds vs. 10.25 seconds), which is demonstrated by the narrower black bars in Figure 9b. Second, eDiscovery increases the intervals of device discovery much faster than STAR when there are few peers and decreases the intervals

⁹The population density of Singapore is very high (7,148/km² vs. 32/km² for the US).

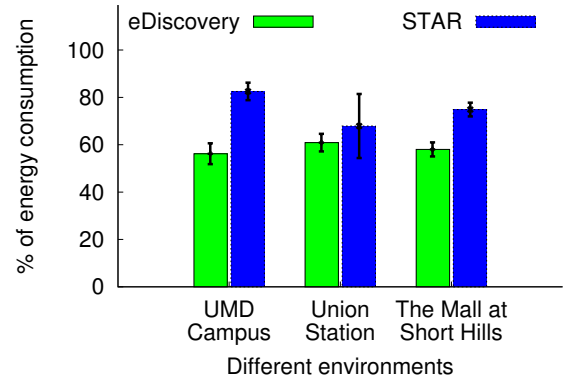


Fig. 8: The comparison of the percentage of energy consumption for different schemes, using Constant as baseline.

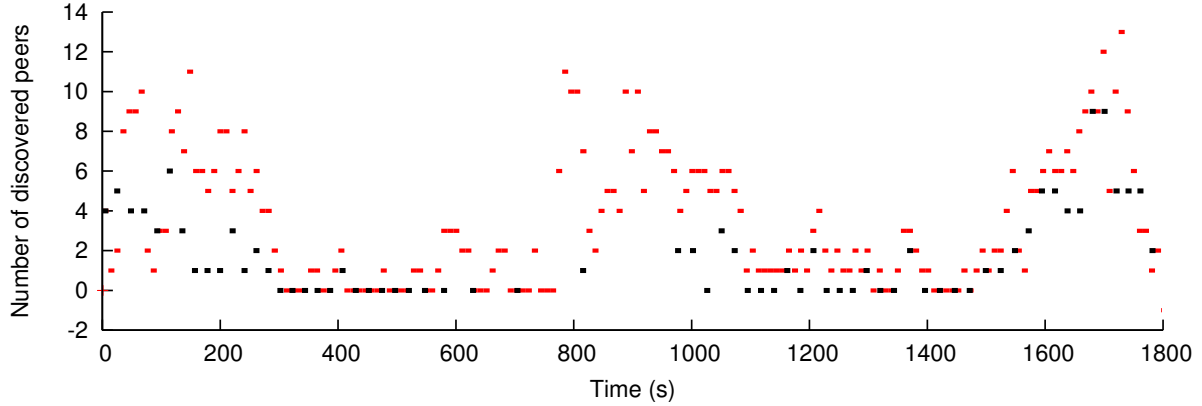
much quicker when there are more peers. For example, from 300 seconds to 600 seconds of both experiments, there were at most 3 peers found by each Bluetooth device discovery. During this quiet period, eDiscovery performed Bluetooth inquiry only 10 times, 3 times less than STAR. Moreover, during the period from 800 seconds to 1,000 seconds when there were more peers, eDiscovery performed Bluetooth inquiry 7 times, 4 times more than STAR. On the one hand, the shorter discovery duration and less frequent Bluetooth device discovery during the quiet period translate into less energy consumption of eDiscovery than STAR. On the other hand, the more frequent device discovery when there are many peers is one of the reasons that the discovery-missing probability of eDiscovery is lower than STAR.

D. Comparison with Another Protocol

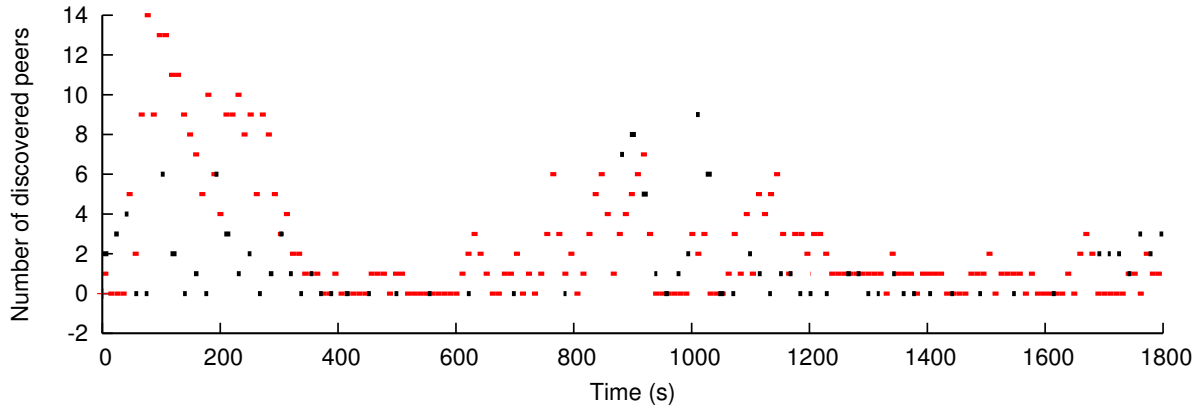
We also evaluate the performance of another protocol RAL proposed by Drula et al. [6]. RAL can discover only less than 30% of peers found in the ground truth for the experiment we did in the Mall at Short Hills. The possible reason may be that even for the most aggressive discovery mode in RAL, the duration of Bluetooth device discovery is less than 1 second, which is too short to complete a scan of all possible Bluetooth frequency bands. Differently from RAL, the shortest duration of Bluetooth device discovery in eDiscovery is 5.12 seconds, which is more suitable when the number of neighboring peers changes dynamically.

E. Summary

To summarize, our performance evaluation shows that if energy consumption is not a major concern and the key objective is to discover more neighboring devices, Constant may be a good choice. It can discover more than 80% peers but consumes only half energy of continuous device discovery. However, when the major goal is to save energy on smartphones and the missing of some peers is acceptable, we should use eDiscovery to dynamically tune the parameters of Bluetooth device discovery.



(a) The start time, duration and the number of discovered peers of Bluetooth device discovery for a STAR experiment. The red horizontal bars are for the ground truth and the black ones are for STAR.



(b) The start time, duration and the number of discovered peers of Bluetooth device discovery for an eDiscovery experiment. The red horizontal bars are for the ground truth and the black ones are for eDiscovery.

Fig. 9: Detailed traces of eDiscovery and STAR experiments.

VII. DISCUSSION

In this section, we discuss the limitations of this paper and some possible extensions of eDiscovery.

A. Limitations

The major limitation of our work presented in this paper is that we have evaluated only a few scenarios: one smartphone and three different environments. We are planning to port eDiscovery to other smartphone platforms, such as Android and iPhones, and evaluate its performance on them.

Although we evaluated the performance of eDiscovery in three different realistic environments, another limitation of the evaluation is that we had no control of other mobile phones during the experiments. If all the mobile phones perform Bluetooth device discovery, the number of phones discovered by us may be changed, as Bluetooth devices that are in inquiry state at the same time cannot discover each other [12], [19]. During our field experiments, most of the discovered Bluetooth devices were probably in discoverable mode only. Running experiments on mobile testbeds, such as CrowdLab [5], may solve this problem.

B. Extensions

Device discovery is only the first step of opportunistic communications. The next two steps are service discovery and data transfer. There are several options of service discovery. We can exploit the standard service discovery protocol of Bluetooth [2], or develop our own protocols.

We plan to leverage multiple radio interfaces on smartphones, such as Bluetooth and WiFi, for opportunistic data transfer. These interfaces usually have different communication ranges and diverse radio characteristics. Pering et al. [18] have demonstrated the benefits of energy reduction by switching between these interfaces for mobile applications. In our case, Bluetooth may be suitable for short data transfer due to its low-power nature. For transmissions of large amounts of data, WiFi may be more desirable, because its data rate is higher and its communication range is much longer than Bluetooth. Although WiFi is not energy efficient for device discovery, we can still enable it for data transfer after mobile phones discover each other through Bluetooth.

C. Other Technologies

There have been other technologies proposed especially to perform device discovery. For example, FlashLinQ [26] is a synchronous wireless PHY/MAC network architecture developed by Qualcomm for direct device-to-device communication over licensed spectrum. It aims to support various applications of proximate Internet, including social networking and mobile advertising. FlashLinQ enables automatic and continuous device discovery and peer-to-peer communication between mobile devices. Although FlashLinQ may be more energy efficient than Bluetooth and WiFi, given its clean-slate design for ad hoc networks, it requires special purpose hardware and also operates in licensed spectrum. Differently from FlashLinQ, we aim to design and implement device discovery protocols using existing hardware and communication technologies available on commercial smartphones.

VIII. CONCLUSION

In this paper, we present eDiscovery, an adaptive device discovery protocol for reducing energy consumption of smartphone-based opportunistic communications. To choose the underlying communication technology, we measured the power of Bluetooth and WiFi device discovery on Nokia N900 and HTC Hero smartphones. Based on the measurement results, we prefer Bluetooth to WiFi because Bluetooth is more energy efficient for device discovery. eDiscovery dynamically changes the Bluetooth inquiry duration and interval to adapt to dynamic environments. We verify the effectiveness of eDiscovery through the first experimental field study of Bluetooth device discovery in three different environments, using a prototype implementation on smartphones. We are currently working on a more extensive evaluation of eDiscovery to further improve its performance.

ACKNOWLEDGEMENT

We thank the anonymous reviewers for their insightful comments. We thank Min Mao and Yu Xiao for their help on measurement setup. We also thank Pan Hui for his support with devices and equipment. The authors were supported in part by US National Science Foundation (NSF) ITR Award CNS-0426683, NSF Award CNS-0626636, and NSF Award CNS 1010789. This work was done when Bo Han was a graduate student at the University of Maryland.

REFERENCES

- [1] L. Aalto, N. Göthlin, J. Korhonen, and T. Ojala. Bluetooth and WAP Push Based Location-Aware Mobile Advertising System. In *Proceedings of MobiSys 2004*, pages 49–58, June 2004.
- [2] Bluetooth Special Interest Group. Specification of the Bluetooth System, Version 2.1 + EDR, 2007.
- [3] G. Chakraborty, K. Naik, D. Chakraborty, N. Shiratori, and D. Wei. Analysis of the Bluetooth device discovery protocol. *Wireless Networks*, 16(2):421–436, Feb. 2010.
- [4] M. Conti, S. Giordano, M. May, and A. Passarella. From Opportunistic Networks to Opportunistic Computing. *IEEE Communications Magazine*, 48(9):126–139, Sept. 2010.
- [5] E. Cuervo, P. Gilbert, B. Wu, and L. P. Cox. CrowdLab: An Architecture for Volunteer Mobile Testbeds. In *Proceedings of COMSNETS 2011*, pages 1–10, Jan. 2011.
- [6] C. Drula, C. Amza, F. Rousseau, and A. Duda. Adaptive Energy Conserving Algorithms for Neighbor Discovery in Opportunistic Bluetooth Networks. *IEEE Journal on Selected Areas in Communications*, 25(1):96–107, Jan. 2007.
- [7] P. Dutta and D. Culler. Practical Asynchronous Neighbor Discovery and Rendezvous for Mobile Sensing Applications. In *Proceedings of SenSys 2008*, pages 71–83, Nov. 2008.
- [8] R. Friedman, A. Kogan, and Y. Krivolapov. On Power and Throughput Tradeoffs of WiFi and Bluetooth in Smartphones. In *Proceedings of INFOCOM 2011*, pages 900–908, Apr. 2011.
- [9] A. Garyfalos and K. C. Almeroth. Coupons: A Multilevel Incentive Scheme for Information Dissemination in Mobile Networks. *IEEE Transactions on Mobile Computing*, 7(6):792–804, June 2008.
- [10] S. Gollakota, F. Adib, D. Katabi, and S. Seshan. Clearing the RF Smog: Making 802.11 Robust to Cross-Technology Interference. In *Proceedings of SIGCOMM 2011*, pages 170–181, Aug. 2011.
- [11] M. Grossglauser and D. N. C. Tse. Mobility Increases the Capacity of Ad Hoc Wireless Networks. *IEEE/ACM Transactions on Networking*, 10(4):477–486, Aug. 2002.
- [12] B. Han, P. Hui, V. S. A. Kumar, M. V. Marathe, J. Shao, and A. Srinivasan. Mobile Data Offloading through Opportunistic Communications and Social Participation. *IEEE Transactions on Mobile Computing*, 11(5):821–834, May 2012.
- [13] D. B. Johnson and D. A. Maltz. *Mobile Computing*, chapter Dynamic Source Routing in Ad Hoc Wireless Networks, pages 153–181. Kluwer Academic Publishers, 1996.
- [14] A. Kandhalu, K. Lakshmanan, and R. R. Rajkumar. U-Connect: A Low-Latency Energy-Efficient Asynchronous Neighbor Discovery Protocol. In *Proceedings of IPSN 2010*, pages 350–361, Apr. 2010.
- [15] M. Liberatore, B. N. Levine, and C. Barakat. Maximizing Transfer Opportunities in Bluetooth DTNs. In *Proceedings of CoNEXT 2006*, pages 1–11, Dec. 2006.
- [16] M. J. McGlynn and S. A. Borbash. Birthday Protocols for Low Energy Deployment and Flexible Neighbor Discovery in Ad Hoc Wireless Networks. In *Proceedings of MOBIHOC 2001*, pages 137–145, Oct. 2001.
- [17] L. McNamara, C. Mascolo, and L. Capra. Media Sharing based on Colocation Prediction in Urban Transport. In *Proceedings of MOBICOM 2008*, pages 58–69, Sept. 2008.
- [18] T. Pering, Y. Agarwal, R. Gupta, and R. Want. CoolSpots: Reducing the Power Consumption of Wireless Mobile Devices with Multiple Radio Interfaces. In *Proceedings of MobiSys 2006*, pages 220–232, June 2006.
- [19] B. S. Peterson, R. O. Baldwin, and J. P. Kharoufeh. Bluetooth Inquiry Time Characterization and Selection. *IEEE Transactions on Mobile Computing*, 5(9):1173–1187, Sept. 2006.
- [20] N. Ristanovic, G. Theodorakopoulos, and J.-Y. L. Boudec. Traps and Pitfalls of Using Contact Traces in Performance Studies of Opportunistic Networks. In *Proceedings of INFOCOM 2012*, pages 1377–1385, Mar. 2012.
- [21] T. Salonidis, P. Bhagwat, and L. Tassiulas. Proximity Awareness and Fast Connection Establishment in Bluetooth. In *Proceedings of MOBIHOC 2000*, pages 141–142, Aug. 2000.
- [22] A. Sharma, V. Navda, R. Ramjee, V. N. Padmanabhan, and E. M. Belding. Cool-Tether: Energy Efficient On-the-fly WiFi Hot-spots using Mobile Phones. In *Proceedings of CoNEXT 2009*, pages 109–120, Dec. 2009.
- [23] T. J. Smith, S. Saroiu, and A. Wolman. BlueMonarch: A System for Evaluating Bluetooth Applications in the Wild. In *Proceedings of MobiSys 2009*, pages 41–53, June 2009.
- [24] S. Vasudevan, D. Towsley, D. Goeckel, and R. Khalili. Neighbor Discovery in Wireless Networks and the Coupon Collector’s Problem. In *Proceedings of MOBICOM 2009*, pages 181–192, Sept. 2009.
- [25] W. Wang, V. Srinivasan, and M. Motani. Adaptive Contact Probing Mechanisms for Delay Tolerant Applications. In *Proceedings of MOBI-COM 2007*, pages 230–241, Sept. 2007.
- [26] X. Wu, S. Tavildar, S. Shakkottai, T. Richardson, J. Li, R. Laroia, and A. Jovicic. FlashLinQ: A Synchronous Distributed Scheduler for Peer-to-Peer Ad Hoc Networks. In *Proceedings of Allerton 2010*, pages 514–521, Sept.-Oct. 2010.
- [27] W. Zhao, M. Ammar, and E. Zegura. A Message Ferrying Approach for Data Delivery in Sparse Mobile Ad Hoc Networks. In *Proceedings of MOBIHOC 2004*, pages 187–198, May 2004.